

```
In [ ]: # 5.1 Градиентный бустинг (классификация).
```

```
In [4]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
```

```
In [ ]: ## 1. Загрузка и обработка датасета.
Дататсет, его загрузка и обработка как в заданиях 1.1, 2.1, 3.1, 4.1.
```

```
In [5]: df = pd.read_csv('C:/Users/Elizaveta/Downloads/magic_gamma/telescope_data.csv')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
df.head(5)
df.info()
print("размер", df.shape)
print("названия столбцов", list(df.columns))
print("уникальные значения в class", df['class'].unique())
print("\nраспределение классов")
print(df['class'].value_counts())
print("доли классов")
print(df['class'].value_counts(normalize=True))
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 19020 entries, 0 to 19019
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Unnamed: 0   19020 non-null  int64
1   fLength      19020 non-null  float64
2   fWidth       19020 non-null  float64
3   fSize        19020 non-null  float64
4   fConc        19020 non-null  float64
5   fConc1       19020 non-null  float64
6   fAsym        19020 non-null  float64
7   fM3Long      19020 non-null  float64
8   fM3Trans     19020 non-null  float64
9   fAlpha       19020 non-null  float64
10  fDist        19020 non-null  float64
11  class        19020 non-null  object
dtypes: float64(10), int64(1), object(1)
memory usage: 1.7+ MB
размер (19020, 12)
названия столбцов ['Unnamed: 0', 'fLength', 'fWidth', 'fSize', 'fConc', 'fConc1',
'fAsym', 'fM3Long', 'fM3Trans', 'fAlpha', 'fDist', 'class']
уникальные значения в class ['g' 'h']
```

распределение классов

class

g 12332

h 6688

Name: count, dtype: int64

доли классов

class

g 0.64837

h 0.35163

Name: proportion, dtype: float64

```
In [6]: df['class']=df['class'].replace({'g': 1, 'h': 0})
print("first 10 values", df['class'].head(10))
print("unique values", df['class'].unique())
print("value counts", df['class'].value_counts())
print("type of data in class", df['class'].dtype)
print("first value", repr(df['class'].iloc[0]))
print("NaN", df['class'].isna().sum())
df=df.drop('Unnamed: 0', axis=1)
```

```

first 10 values 0    1
1    1
2    1
3    1
4    1
5    1
6    1
7    1
8    1
9    1
Name: class, dtype: int64
unique values [1 0]
value counts class
1    12332
0     6688
Name: count, dtype: int64
type of data in class int64
first value np.int64(1)
NaN 0

```

C:\Users\Elizaveta\AppData\Local\Temp\ipykernel_22604\2620301820.py:1: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly call `result.infer\_objects(copy=False)`. To opt-in to the future behavior, set `pd.set\_option('future.no\_silent\_downcasting', True)`

```
df['class']=df['class'].replace({'g': 1, 'h': 0})
```

```

In [7]: X=df.drop('class', axis=1)
        Y=df['class']
        print("size of X", {X.shape})
        print("size of Y", {Y.shape})
        X_train, X_test, Y_train, Y_test = train_test_split(
            X, Y,
            test_size=0.3,
            random_state = 42,
            stratify = Y)
        print(f"size of X_train", {X_train.shape})
        print(f"size of X_test", {X_test.shape})
        print(f"size of Y_train", {Y_train.shape})
        print(f"size of Y_test", {Y_test.shape})
        print("balance of classes:")
        print(f"start data: 1={Y.mean():.3f}, 0={1-Y.mean():.3f}")
        print(f"train data: 1={Y_train.mean():.3f}, 0={1-Y_train.mean():.3f}")
        print(f"start data: 1={Y_test.mean():.3f}, 0={1-Y_test.mean():.3f}")

```

```

size of X {(19020, 10)}
size of Y {(19020,)}
size of X_train {(13314, 10)}
size of X_test {(5706, 10)}
size of Y_train {(13314,)}
size of Y_test {(5706,)}
balance of classes:
start data: 1=0.648, 0=0.352
train data: 1=0.648, 0=0.352
start data: 1=0.648, 0=0.352

```

```

In [ ]: # 2. Создание бейзлайна.
        Обучение модели RandomForestClassifier из библиотеки sklearn.
        F1 учитывает оба класса и их ошибки, показывает точность (сколько из отнесённых
        и полноту (сколько найдено настоящих объектов класса) классификации. В случае, е
        при высоком Accurasy, F1 покажет это.

```

Accuracy показывает, какую долю объектов модель классифицировала правильно. В да поэтому эта метрика даст адекватную оценку.
 Обучающая выборка (train) используется для обучения модели.
 Тестовая выборка (test) используется для оценки качества обученной модели.
 Результат на бейзлайне: F1=0.9093, accuracy=0.8766.
 Получены достаточно хорошие результаты, но есть потенциал для улучшения. Эти дан отталкиваться в сторону улучшений, показывающей, чего можно достичь минимальными

```
In [24]: from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import f1_score, accuracy_score
baseline_gb = GradientBoostingClassifier(random_state=42)
baseline_gb.fit(X_train, Y_train)
Y_baseline_gb = baseline_gb.predict(X_test)
F1 = f1_score(Y_test, Y_baseline_gb)
accuracy = accuracy_score(Y_test, Y_baseline_gb)
print(f" F1={F1:.4f}, accuracy={accuracy:.4f}")
```

F1=0.9093, accuracy=0.8766

```
In [ ]: ## 3. Улучшение бейзлайна.
Гипотеза 1: Подбор гиперпараметров.
С помощью RandomSearchCV подберём количество деревьев в ансамбле, шаг градиентно
минимальное число объектов в узлах и листьях, долю выборки, используемой для обу
разбиении. Выбирается 50 случайных комбинаций. Результаты:
Лучшие параметры: {'subsample': 1.0, 'n_estimators': 500, 'min_samples_split': 1
                    'learning_rate': 0.05}
Лучший F1 на CV: 0.9099058460631213
Улучшенный классификатор: F1 = 0.9165, accuracy = 0.8882.
Полученные результаты говорят об улучшении качества модели.
```

```
In [26]: from sklearn.model_selection import RandomizedSearchCV
param_dist_clf = {
    'n_estimators': [100, 200, 300, 500],
    'learning_rate': [0.01, 0.05, 0.1, 0.2],
    'max_depth': [3, 5, 7],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 5],
    'subsample': [0.8, 1.0],
    'max_features': ['sqrt', 'log2', 0.5, 1.0]
}
gb_clf = GradientBoostingClassifier(random_state=42)
random_search_clf = RandomizedSearchCV(
    gb_clf,
    param_distributions=param_dist_clf,
    n_iter=50,
    cv=5,
    scoring='f1',
    random_state=42,
    n_jobs=-1,
    verbose=1
)
random_search_clf.fit(X_train, Y_train)
print("Лучшие параметры:", random_search_clf.best_params_)
print("Лучший F1 на CV:", random_search_clf.best_score_)
best_gb_clf = random_search_clf.best_estimator_
y_pred = best_gb_clf.predict(X_test)
print(f"Улучшенный классификатор: F1 = {f1_score(Y_test, y_pred):.4f}, accuracy
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits

Лучшие параметры: {'subsample': 1.0, 'n_estimators': 500, 'min_samples_split': 10, 'min_samples_leaf': 2, 'max_features': 'sqrt', 'max_depth': 7, 'learning_rate': 0.05}

Лучший F1 на CV: 0.9099058460631213

Улучшенный классификатор: F1 = 0.9165, accuracy = 0.8882

In []: *## 4. Имплементация алгоритма машинного обучения.*

Был создан собственный алгоритм градиентного бустинга, построенный на основе алг

```
In [18]: class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, value=None):
        self.feature = feature
        self.threshold = threshold
        self.left = left
        self.right = right
        self.value = value

    class MyDecisionTreeRegressor:
        def __init__(self, max_depth=None, min_samples_split=2, min_samples_leaf=1,
                     random_state=None):
            self.max_depth = max_depth
            self.min_samples_split = min_samples_split
            self.min_samples_leaf = min_samples_leaf
            self.random_state = random_state
            self.root = None
            self.n_features_ = None
            self.rng = np.random.RandomState(random_state)

        def fit(self, X, y):
            X = np.array(X)
            y = np.array(y)
            self.n_features_ = X.shape[1]
            self.root = self._build_tree(X, y, depth=0)
            return self

        def _build_tree(self, X, y, depth):
            n_samples = X.shape[0]
            if (depth == self.max_depth) or (n_samples < self.min_samples_split) or \
                (n_samples < self.min_samples_leaf):
                leaf_value = np.mean(y)
                return Node(value=leaf_value)
            best_feat, best_thresh, best_gain = self._best_split(X, y)
            if best_feat is None:
                leaf_value = np.mean(y)
                return Node(value=leaf_value)
            left_mask = X[:, best_feat] <= best_thresh
            right_mask = ~left_mask
            left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
            right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)
            return Node(feature=best_feat, threshold=best_thresh, left=left_child, right=right_child)

        def _best_split(self, X, y):
            best_gain = -float('inf')
            best_feature = None
            best_threshold = None
            current_var = np.var(y)
            n_features = X.shape[1]
            for feature in range(n_features):
                thresholds = np.unique(X[:, feature])
                for thresh in thresholds:
                    left_mask = X[:, feature] <= thresh
                    right_mask = ~left_mask
                    if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mask) < self.min_samples_leaf:
                        continue
                    left_var = np.var(y[left_mask])
                    right_var = np.var(y[right_mask])
                    gain = current_var - (np.sum(left_mask) * left_var + np.sum(right_mask) * right_var)
                    if gain > best_gain:
                        best_gain = gain
                        best_feature = feature
                        best_threshold = thresh
```

```

        right_var = np.var(y[right_mask])
        n_left = np.sum(left_mask)
        n_right = np.sum(right_mask)
        n_total = n_left + n_right
        gain = current_var - (n_left / n_total) * left_var - (n_right /
        if gain > best_gain:
            best_gain = gain
            best_feature = feature
            best_threshold = thresh
    if best_gain <= 0:
        return None, None, None
    return best_feature, best_threshold, best_gain
def predict(self, X):
    X = np.array(X)
    predictions = []
    for x in X:
        node = self.root
        while node.value is None:
            if x[node.feature] <= node.threshold:
                node = node.left
            else:
                node = node.right
        predictions.append(node.value)
    return np.array(predictions)

```

```

In [19]: class MyGradientBoostingClassifier:
    def __init__(self, n_estimators=100, learning_rate=0.1, max_depth=3,
                  min_samples_split=2, min_samples_leaf=1, subsample=1.0,
                  random_state=None):
        self.n_estimators = n_estimators
        self.learning_rate = learning_rate
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.subsample = subsample
        self.random_state = random_state
        self.trees = []
        self.initial_pred_ = None
        self.rng = np.random.RandomState(random_state)
    def _sigmoid(self, F):
        F = np.clip(F, -500, 500)
        return 1 / (1 + np.exp(-F))
    def fit(self, X, y):
        X = np.array(X)
        y = np.array(y).astype(float)
        p0 = np.mean(y)
        self.initial_pred_ = np.log(p0 / (1 - p0 + 1e-15))
        F = np.full(len(y), self.initial_pred_)
        for i in range(self.n_estimators):
            p = self._sigmoid(F)
            residuals = y - p
            if self.subsample < 1.0:
                n_samples = int(self.subsample * len(y))
                indices = self.rng.choice(len(y), size=n_samples, replace=False)
                X_sub = X[indices]
                residuals_sub = residuals[indices]
            else:
                X_sub = X
                residuals_sub = residuals
            tree = MyDecisionTreeRegressor(

```

```

        max_depth=self.max_depth,
        min_samples_split=self.min_samples_split,
        min_samples_leaf=self.min_samples_leaf,
        random_state=self.rng.randint(0, 10**6)
    )
    tree.fit(X_sub, residuals_sub)
    self.trees.append(tree)
    F += self.learning_rate * tree.predict(X)
    return self
def predict_proba(self, X):
    X = np.array(X)
    F = np.full(len(X), self.initial_pred_)
    for tree in self.trees:
        F += self.learning_rate * tree.predict(X)
    p = self._sigmoid(F)
    return np.column_stack((1 - p, p))
def predict(self, X, threshold=0.5):
    proba = self.predict_proba(X)[: , 1]
    return (proba >= threshold).astype(int)

```

In []: Тестирование собственного алгоритма градиентного бустинга. Результаты:
 F1 = 0.9087, Accuracy = 0.8709
 Полученные результаты подтверждают корректность собственного алгоритма.

```

In [ ]: my_gb_clf = MyGradientBoostingClassifier(
    n_estimators=200,
    learning_rate=0.05,
    max_depth=3,
    min_samples_split=2,
    min_samples_leaf=1,
    subsample=0.8,
    random_state=42
)
my_gb_clf.fit(X_train, Y_train)
y_pred = my_gb_clf.predict(X_test)
print("MyGradientBoostingClassifier")
print(f"F1 = {f1_score(Y_test, y_pred):.4f}")
print(f"Accuracy = {accuracy_score(Y_test, y_pred):.4f}")

```

In []: ## Выводы:
 При исследовании градиентного бустинга была обучена базовая модель с высокими результатами. Были подобраны гиперпараметры, благодаря которым удалось повысить метрики до F1 на основе решающего дерева из предыдущего задания, результат его работы составил алгоритма.